

Coding Fluency- Workflows & Best Practices



Suman

22 Jan, 2026 At 8:00 PM

Notes

Trimester-I

Recommended

Resource



Associated Lectures (1)



Description



Discussions

SESSION 1.3 LECTURE NOTES

Coding Fluency: Workflows & Best Practices

In Class Notes

Mental Map

SESSION 1.3: CONTROL FLOW, CONDITIONALS & LOOPS

- REVISION (from 1.2)
 - Type conversion: `int()`, `float()`, `str()`
 - Arithmetic: `+` `-` `*` `/` `//` `%` `**`
 - String access: `text[0]`, `text[1:4]`
- STRING METHODS
 - Case: `.upper()`, `.lower()`, `.title()`
 - Whitespace: `.strip()`, `.lstrip()`, `.rstrip()`
 - Search: `.find()`, `.count()`, `.startswith()`
 - Modify: `.replace()`, `.split()`, `.join()`
- F-STRINGS & FORMATTING
 - Basic: `f"Hello, {name}"`
 - Decimals: `f"{value:.2f}"`
 - Alignment: `f"{text:^20}"`
 - Thousands: `f"{num:,}"`
- INPUT/OUTPUT
 - `print(sep=, end=)`
 - `input()` → ALWAYS string!
- COMPARISON OPERATORS
 - `==` `!=` `<` `>` `<=` `>=`
 - `=` (assign) vs `==` (compare)

LOGICAL OPERATORS

- and: both must be True
- or: at least one True
- not: flip the value

CONDITIONALS

- if: single condition
- if/else: two paths
- if/elif/else: multiple paths
- Nested: decisions within decisions

FOR LOOPS

- range(n): 0 to n-1
- range(start, stop): start to stop-1
- range(start, stop, step): with increment

WHILE LOOPS

- while condition: repeat until False
- while True: infinite until break

LOOP CONTROL

- break: exit loop immediately
- continue: skip to next iteration
- Patterns: counter, accumulator, search

The Big Picture

This session transforms you from writing simple calculations to building **smart programs** that can make decisions and repeat actions. By the end, you will have built a complete Number Guessing Game.

Why This Matters for AI/ML:

- Data processing requires loops to iterate through datasets
- Conditionals filter and classify data
- Input validation ensures clean data entry
- These patterns form the foundation of every algorithm

Quick Reference Tables

String Methods

Method	Example	Result	Purpose
<code>.upper()</code>	<code>"hello".upper()</code>	<code>"HELLO"</code>	All uppercase
<code>.lower()</code>	<code>"HELLO".lower()</code>	<code>"hello"</code>	All lowercase
<code>.title()</code>	<code>"hello world".title()</code>	<code>"Hello World"</code>	Title case

Method	Example	Result	Purpose
<code>.capitalize()</code>	<code>"hello world".capitalize()</code>	<code>"Hello world"</code>	First char only
<code>.strip()</code>	<code>" hi ".strip()</code>	<code>"hi"</code>	Remove whitespace (both ends)
<code>.lstrip()</code>	<code>" hi ".lstrip()</code>	<code>"hi "</code>	Remove left whitespace
<code>.rstrip()</code>	<code>" hi ".rstrip()</code>	<code>" hi"</code>	Remove right whitespace
<code>.find(s)</code>	<code>"hello".find("l")</code>	<code>2</code>	Find position (-1 if not found)
<code>.count(s)</code>	<code>"hello".count("l")</code>	<code>2</code>	Count occurrences
<code>.startswith(s)</code>	<code>"hello".startswith("he")</code>	<code>True</code>	Check prefix
<code>.endswith(s)</code>	<code>"hello".endswith("lo")</code>	<code>True</code>	Check suffix
<code>.replace(a,b)</code>	<code>"cat".replace("c","b")</code>	<code>"bat"</code>	Replace text
<code>.split(sep)</code>	<code>"a,b,c".split(",")</code>	<code>["a","b","c"]</code>	Break into list
<code>.join(list)</code>	<code>"-".join(["a","b"])</code>	<code>"a-b"</code>	Combine list into string

Key Point: Strings are IMMUTABLE. Methods return NEW strings; they do not modify the original.

F-String Formatting

Basic Usage

```
name = "Raj"
age = 22
print(f"Hello, {name}!")      # Hello, Raj!
print(f"Next year: {age + 1}") # Next year: 23
```

Number Formatting

Format	Example	Result	Meaning
<code>{x}</code>	<code>f"{3.14159}"</code>	<code>3.14159</code>	Default
<code>{x:.2f}</code>	<code>f"{3.14159:.2f}"</code>	<code>3.14</code>	2 decimal places
<code>{x:.1f}</code>	<code>f"{3.14159:.1f}"</code>	<code>3.1</code>	1 decimal place
<code>{x:.0f}</code>	<code>f"{3.14159:.0f}"</code>	<code>3</code>	0 decimal places
<code>{x:,}</code>	<code>f"{1000000:,}"</code>	<code>1,000,000</code>	Thousands separator

Format	Example	Result	Meaning
{x:,.2f}	f"{1234.5:,.2f}"	1,234.50	Both
{x:.2%}	f"{0.0087:.2%}"	0.87%	Percentage

Alignment

Format	Example	Result	Meaning
{s:10}	f"{'Hi':10}"	'Hi '	Left-align (default)
{s:>10}	f"{'Hi':>10}"	' Hi '	Right-align
{s:^10}	f"{'Hi':^10}"	' Hi '	Center

Input/Output Patterns

print() Options

```
print("A", "B", "C")           # A B C (default space separator)
print("A", "B", "C", sep="-") # A-B-C (custom separator)
print("Loading", end="")       # No newline at end
print("...")                   # Continues on same line
```

input() Pattern (CRITICAL!)

```
# Text input
name = input("Name: ")

# Number input - MUST CONVERT!
age = int(input("Age: "))
height = float(input("Height: "))

# Clean text input
name = input("Name: ").strip().title()
```

Remember: `input()` **ALWAYS** returns a string!

Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal	5 != 3	True
>	Greater than	5 > 3	True

Operator	Meaning	Example	Result
<	Less than	5 < 3	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	5 <= 3	False

CRITICAL DISTINCTION:

- = is **assignment** (put value into variable)
- == is **comparison** (check if equal)

Logical Operators

Operator	Rule	Example	Result
and	Both must be True	True and False	False
or	At least one True	True or False	True
not	Flip the value	not True	False

Truth Tables**AND (both must be True):**

True and True → True
 True and False → False
 False and True → False
 False and False → False

OR (at least one True):

True or True → True
 True or False → True
 False or True → True
 False or False → False

NOT (flip):

not True → False
 not False → True

Conditional Statements

Basic if

```
if condition:
    # runs if condition is True
```

if/else

```
if condition:
    # runs if True
else:
    # runs if False
```

if/elif/else

```
if condition1:
    # first match
elif condition2:
    # second option
elif condition3:
    # third option
else:
    # default (no match)
```

Nested Conditionals

```
if outer_condition:
    if inner_condition:
        # both conditions true
    else:
        # outer true, inner false
else:
    # outer false
```

for Loops with range()

range() Variations

Expression	Values Generated	Count
<code>range(5)</code>	0, 1, 2, 3, 4	5 values
<code>range(1, 6)</code>	1, 2, 3, 4, 5	5 values
<code>range(0, 10, 2)</code>	0, 2, 4, 6, 8	5 values
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1	5 values

Basic for Loop

```
for i in range(5):  
    print(i) # Prints 0, 1, 2, 3, 4
```

Counting from 1

```
for i in range(1, 6):  
    print(i) # Prints 1, 2, 3, 4, 5
```

Countdown

```
for i in range(5, 0, -1):  
    print(i) # Prints 5, 4, 3, 2, 1
```

while Loops

Basic while

```
count = 1  
while count <= 5:  
    print(count)  
    count = count + 1 # MUST change to avoid infinite loop!
```

while True with break

```
while True:  
    user_input = input("Enter (q to quit): ")  
    if user_input == "q":  
        break # Exit the loop  
    print(f"You entered: {user_input}")
```

for vs while

Use for when...	Use while when...
You know how many iterations	You do not know how many iterations
Iterating through a collection	Waiting for a condition to change
Counting a specific range	Running until user quits

Loop Control: break and continue

break - Exit Loop Immediately

```
for i in range(10):
    if i == 5:
        break # Stops loop entirely
    print(i)
# Output: 0, 1, 2, 3, 4
```

continue - Skip to Next Iteration

```
for i in range(5):
    if i == 2:
        continue # Skips printing 2
    print(i)
# Output: 0, 1, 3, 4
```

Key Frameworks

Framework 1: Input-Validate-Process Pattern

```
user_input = input("Enter a number: ")
value = int(user_input) # Convert

if value >= 1 and value <= 100: # Validate
    # Process valid input
    result = value * 2
    print(f"Result: {result}")
else:
    print("Invalid! Must be 1-100.")
```

Framework 2: Counter Pattern

```
count = 0
for item in collection:
    if condition:
        count = count + 1
print(f"Found {count} items")
```

Framework 3: Accumulator Pattern

```
total = 0
for num in numbers:
    total = total + num
print(f"Sum: {total}")
```

Framework 4: Search Pattern

```
found = False
for item in collection:
    if item == target:
        found = True
        break
if found:
    print("Found it!")
```

Framework 5: Game Loop Pattern

```
while True:
    # Get user input
    guess = int(input("Your guess: "))

    # Validate
    if guess < 1 or guess > 100:
        print("Invalid!")
        continue

    # Check win condition
    if guess == secret:
        print("You win!")
        break

    # Give feedback
    elif guess < secret:
        print("Too low!")
    else:
        print("Too high!")
```

Common Errors Quick Reference

Error	Symptom	Cause	Fix
IndentationError	"expected an indented block"	Missing indent after if/for/while	Add 4 spaces
Infinite Loop	Program never stops	Condition never becomes False	Update variable in loop
TypeError	"can only concatenate str..."	Forgot to convert input	Use <code>int(input(...))</code>
Off-by-one	Getting 0-4 instead of 1-5	<code>range()</code> starts at 0	Use <code>range(1, 6)</code>
SyntaxError	In condition	Used <code>=</code> instead of <code>==</code>	Use <code>==</code> for comparison
Wrong path taken	if block runs when it should not	Logic error in condition	Check operator and values

IndentationError Example

```
# WRONG
if x > 5:
print("big")

# CORRECT
if x > 5:
    print("big")
```

Infinite Loop Example

```
# WRONG (never stops)
count = 0
while count < 5:
    print(count)
    # Forgot count = count + 1

# CORRECT
count = 0
while count < 5:
    print(count)
    count = count + 1
```

TypeError with input() Example

```
# WRONG
age = input("Age: ")
print(age + 1) # Error!

# CORRECT
age = int(input("Age: "))
print(age + 1)
```

Today's Mini-Project: Number Guessing Game

Concepts Used

Concept	Where Used
import	<code>import random</code> for random number
String Methods	<code>.lower()</code> for play again check
F-Strings	<code>f"You got it in {attempts} attempts!"</code>
I/O	<code>print()</code> and <code>input()</code> for interaction
Comparison	<code>guess == secret</code> , <code>guess < secret</code>

Concept	Where Used
Logical	<code>guess >= 1 and guess <= 100</code>
if/elif/else	Checking correct/too high/too low
while loop	Keep playing until correct
break	Exit when guess is correct
continue	Skip invalid guesses

Simplified Game Code

```
import random

secret = random.randint(1, 100)
attempts = 0

print("Guess the number (1-100)")

while True:
    guess = int(input("Your guess: "))
    attempts = attempts + 1

    if guess == secret:
        print(f"Correct in {attempts} attempts!")
        break
    elif guess < secret:
        print("Too low!")
    else:
        print("Too high!")
```

Key Takeaways

Topic	Key Point
String Methods	Methods return NEW strings (immutable)
<code>.strip()</code>	Essential for cleaning user input
F-Strings	Use <code>:.2f</code> for 2 decimal places
<code>input()</code>	ALWAYS returns a string - convert for numbers
Comparison	<code>==</code> compares, <code>=</code> assigns
Logical	<code>and</code> = both, <code>or</code> = either, <code>not</code> = flip
if/elif/else	First True condition wins, order matters
for loop	Use when you know how many iterations

Topic	Key Point
while loop	Use when you do not know how many iterations
break	Exits loop immediately
continue	Skips to next iteration
Infinite loop danger	Always ensure condition can become False

Next Session Preview

Session 2.1: Data Structures I - Lists & Tuples

- What if we want to store multiple values together?
- Lists: ordered, mutable collections
- Tuples: ordered, immutable collections
- List operations: indexing, slicing, append, remove
- Project: Build a Todo List Manager!

```
# Preview: Lists
scores = [3, 5, 2, 7, 4]
print(f"Best score: {min(scores)}")
print(f"Average: {sum(scores) / len(scores)}")
print(f"Total games: {len(scores)}")
```

Vishlesan i-Hub IIT Patna × Masai School